Feuille de route du projet

SAÉ BUT3 5.Real.01 – Application web découplée de recommandation comparée

Animés · Films · Séries

Équipe « Les Lions Conquérants »
CAV Boran · CHOUAT Oussama · LARIBI Younes · LE MESTE Enzo · TABELLOUT Amir Khaled

Responsable : Gaël Guibon · **Version du document :** 31/10/2026

Backend	Flask (API JSON), blueprints, services, DAO, ORM SQLAlchemy – structure des TD R5.A.05
Frontend	Pages HTML/JS séparées, Bootstrap 5, Chart.js, appels fetch vers l'API
Données	SQLite (PostgreSQL possible en changeant l'URI), API Jikan + TMDB
Reco	Approche 1 : filtrage par contenu · Approche 2 : embeddings + similarité cosinus

1. Échéances imposées

Dates du sujet (celles marquées ~ sont encore à confirmer par les enseignants). Chaque rendu dev correspond à un tag git.

Date	Livrable	Compétence	Tag
08/10	Description du projet (équipe, thème, données, stack)	Dev	`fait`
18/11	Dépôt git : première version (interface complète, controllers + services, reco unique)	Dev	`v0.5.0`
~02/12	PDF : axes d'amélioration de la v1 (vulgarisation, vitesse, batterie, stratégies)	Optim	
10/12	Branding + logo + stratégie de communication intra-équipe (git, outils)	Comm	`v0.7.0`
~09/01	PDF : rapport de performances et calcul des coûts	Optim	
15/01	Version finale + vidéo de démonstration + dépôt git final	Dev	`v1.0.0`
~19/01	Rapport d'optimisation global + rapport de présentation de l'application	Optim / Comm	

2. Planning global (semaines du lundi)

Phase	Tag	26/10	02/11	09/11	16/11	23/11	30/11	07/12	14/12	21/12	28/12	04/01	11/01	18/01
0. Mise en place	`v0.1.0`													
1. Données réelles	`v0.2.0`													
2. Front catalogue	`v0.3.0`													
3. Approche 1 (contenu)	`v0.4.0`													
4. Première version	`v0.5.0`													
5. Approche 2 (embeddings)	`v0.6.0`													
6. Branding & comm	`v0.7.0`													
7. Dashboard complet	`v0.8.0`													
Vacances : tampon														
8. Optimisation	`v0.9.0`													
9. Version finale	`v1.0.0`													
Rendus					18/11		02/12	10/12				09/01	15/01	19/01

Les phases se chevauchent volontairement : pendant que les uns importent les données, les autres avancent le front sur le jeu d'essai. Le sujet conseille de finir une première approche de recommandation avant de commencer la seconde : c'est l'ordre retenu (phases 3 puis 5).

3. Méthode de travail avec git

• **main** est toujours fonctionnelle. Chaque tâche se fait sur une branche courte : `feat/reco-contenu`, `fix/pagination`…

• Chaque branche est fusionnée par **pull request relue par un autre membre** : tout le monde connaît le code des autres, ce qui est indispensable pour l'oral.

• Messages de commit au format `type(portée): description`. Types : **feat** (fonctionnalité), **fix** (correction), **docs**, **test**, **perf** (optimisation), **refactor**, **chore** (outillage, versions).

• Un commit = un changement compréhensible. Mieux vaut 5 petits commits qu'un seul énorme « avancement ».

• Versions au format SemVer : `v0.MINEUR.CORRECTIF` tant que l'application n'est pas finale (+1 en mineur à chaque phase, +1 en correctif pour une correction), `v1.0.0` au rendu final.

```
git checkout -b feat/reco-contenu
git add .
git commit -m "feat(reco): approche 1 – score de similarité par contenu"
git push -u origin feat/reco-contenu   # puis pull request sur GitHub

# à la fin d'une phase, sur main :
git tag -a v0.4.0 -m "approche 1 fonctionnelle"
git push origin v0.4.0
```

4. Détail des phases et des commits

Pour chaque phase : l'objectif, la liste des commits dans l'ordre conseillé, et le critère qui dit que la phase est terminée.

Phase 0 : Mise en place

31/10 – 01/11 · **v0.1.0**

Objectif : Un dépôt partagé où chacun peut lancer le backend et le front en 2 commandes. *(État actuel du projet.)*

#	Message de commit	Contenu
1	`chore: initialisation du dépôt (.gitignore, README racine)`	Arborescence projet/ : backend/, frontend/, docs/.
2	`feat(backend): application factory et blueprints index/catalogue/reco`	create_app(), enregistrement des blueprints sous /api, CORS, erreurs 400/404 en JSON.
3	`feat(backend): modèles ORM Content/Genre, DAO et services`	Base + BaseModelMixin, table d'association N-N, ContentSqliteORM, ContentService, RecoService (méthodes vides).
4	`feat(backend): script prepa_db avec jeu d'essai`	prepa_db.py (7 contenus), requirements.txt, README backend.
5	`feat(frontend): squelette des pages et barre de navigation`	index / catalogue / reco (en construction), js/nav.js, js/api.js, Bootstrap.
6	`docs: feuille de route du projet`	docs/feuille_de_route.pdf (ce document).

Terminé quand : *python main.py + python -m http.server 5500 affichent l'accueil ; /api répond. Tag v0.1.0 poussé.*

Phase 1 : Données réelles

02/11 – 06/11 · **v0.2.0**

Objectif : Remplacer le jeu d'essai par un vrai catalogue (~1 500 contenus : 500 animés, 500 films, 500 séries).

#	Message de commit	Contenu
1	`feat(models): champs source et external_id sur Content`	Origine (jikan / tmdb) + identifiant externe, contrainte d'unicité : pas de doublons si on relance l'import.
2	`feat(data): import des animés depuis l'API Jikan`	scripts/import_jikan.py : top animés paginé, pause entre requêtes (limite ~3 req/s), genres créés à la volée.
3	`feat(data): import des films et séries depuis TMDB`	scripts/import_tmdb.py : /movie/popular, /tv/popular, liste des genres. Clé API dans .env (jamais commitée).
4	`feat(api): recherche par titre et pagination du catalogue`	?q= (ilike), ?page= & ?per_page= ; méthode DAO findByTitle.
5	`docs: choix et ajout de la licence`	Fichier LICENSE + justification (réutilisation, compatibilité avec les conditions TMDB / Jikan).

Terminé quand : ≥ *1 000 contenus en base avec synopsis et genres ; /api/catalogue filtrable et paginé.*

Phase 2 : Front du catalogue

04/11 – 10/11 · **v0.3.0**

Objectif : Pouvoir parcourir tout le catalogue depuis le navigateur (en parallèle de la phase 1, sur le jeu d'essai au début).

#	Message de commit	Contenu
1	`feat(frontend): catalogue en cartes avec filtres type et genre`	catalogue.html + js/catalogue.js : appels /api/catalogue, cartes Bootstrap comme en TD03.
2	`feat(frontend): recherche par titre et pagination`	Champ de recherche, boutons précédent / suivant.
3	`feat(frontend): affiches et fiche détaillée d'un contenu`	Image de couverture, synopsis complet (modal Bootstrap), bouton « Recommandations ».
4	`fix(frontend): états de chargement et d'erreur`	Spinner pendant les appels, message clair si l'API ne répond pas.

Terminé quand : *Recherche, filtres et pagination fonctionnent sur les vraies données.*

Phase 3 : Approche 1 – filtrage par contenu

07/11 – 13/11 · **v0.4.0**

Objectif : Première recommandation complète, comme le conseille le sujet avant d'attaquer la seconde.

#	Message de commit	Contenu
1	`feat(reco): approche 1 – score de similarité par contenu`	RecoService.recoContenu : Jaccard des genres + bonus même type + proximité d'année + note ; top-n.
2	`feat(reco): justification de chaque recommandation`	Chaque résultat renvoie les critères qui ont joué (genres communs…) : répond au « pourquoi cette reco ? ».
3	`test(reco): tests de l'approche 1`	tests/test_reco_contenu.py : jamais le contenu lui-même, n respecté, scores décroissants.
4	`feat(frontend): page Comparer avec l'approche 1`	reco.html + js/reco.js : choix du contenu, liste des résultats et leur justification.
5	`docs(reco): complexité de l'approche 1`	docs/complexite.md : O(N·G) pour les scores + O(N log N) pour le tri.

Terminé quand : *Recommandations cohérentes et expliquées visibles dans l'interface.*

Phase 4 : Première version (rendu 18/11)

12/11 – 18/11 · **v0.5.0**

Objectif : Remplir le jalon : interface complète (pas forcément jolie), controllers + services en place, reco unique fonctionnelle.

#	Message de commit	Contenu
1	`feat(metrics): journalisation des temps de recommandation`	Modèle RecoLog (méthode, content_id, n, time_ms, date) écrit à chaque appel.
2	`feat(api): endpoint de métriques min/max/moyenne`	Blueprint metrics : /api/metrics?method= renvoie min, max, moyenne, nombre d'appels.
3	`feat(frontend): dashboard v1`	Chart.js : temps de réponse par méthode + tableau min / max / moyenne.
4	`feat(frontend): emplacement de l'approche 2 dans la comparaison`	Deuxième colonne « en cours de développement » : l'interface est complète.
5	`docs: README d'installation et de lancement`	venv, import des données, lancement des deux serveurs, captures d'écran.

Terminé quand : *Tag v0.5.0 poussé avant le 18/11. Chaque membre a lancé l'application sur son poste.*

Phase 5 : Approche 2 – embeddings

19/11 – 01/12 · **v0.6.0**

Objectif : Deuxième méthode de recommandation et vraie comparaison côte à côte.

#	Message de commit	Contenu
1	`feat(reco): génération des embeddings des synopsis`	scripts/build_embeddings.py : sentence-transformers (all-MiniLM-L6-v2, 384 dim), calcul hors ligne, sauvegarde .npy.
2	`feat(reco): approche 2 – similarité cosinus`	Matrice normalisée chargée une seule fois ; cosinus = produit scalaire ; top-n.
3	`feat(reco): repli TF-IDF si le modèle n'est pas installé`	TfidfVectorizer (scikit-learn) : pas de dépendance lourde obligatoire sur les postes faibles.
4	`test(reco): tests de l'approche 2`	Mêmes garanties que l'approche 1.
5	`feat(frontend): comparaison côte à côte des deux approches`	Deux colonnes, contenus présents dans les deux listes surlignés.
6	`feat(metrics): recouvrement entre les deux listes`	Indice de Jaccard des résultats affiché dans le dashboard.
7	`docs(reco): complexité de l'approche 2`	O(N·d) par requête, O(N·d) en mémoire, coût du pré-calcul.
8	`docs(optim): rapport des axes d'amélioration (rendu ~02/12)`	Vulgarisation, points lents identifiés, pistes pour la batterie, stratégies envisagées.

Terminé quand : *Les deux approches tournent en direct et sont comparées dans l'interface ; rapport optim rendu.*

Phase 6 : Branding & comm (rendu 10/12)

30/11 – 10/12 · **v0.7.0**

Objectif : Donner une identité à l'application et documenter le fonctionnement de l'équipe.

#	Message de commit	Contenu
1	`feat(frontend): identité visuelle (nom, logo, palette, favicon)`	Logo SVG, feuille de style commune, couleurs dédiées à chaque approche.
2	`feat(frontend): page d'accueil finale avec slogan`	Présentation du concept et accès direct à la comparaison.
3	`docs(comm): stratégie de communication intra-équipe`	Branches, pull requests, relectures, GitHub Projects / Trello, réunions.
4	`docs(comm): présentation du branding`	Choix du nom, du logo, des couleurs et leur motivation.

Terminé quand : *Livrable du 10/12 rendu ; l'application porte son nom et son logo.*

Phase 7 : Dashboard complet

07/12 – 18/12 · **v0.8.0**

Objectif : Comparer à tous les niveaux demandés : résultat, rapidité, énergie, complexité, besoins matériels.

#	Message de commit	Contenu
1	`feat(metrics): mesure de la mémoire et du temps CPU`	tracemalloc + time.process_time en plus du temps réel.
2	`feat(metrics): estimation de la consommation énergétique`	CodeCarbon, ou temps CPU × puissance moyenne du processeur ; en Wh par requête.
3	`feat(api): banc d'essai /api/bench`	Lance k requêtes aléatoires par méthode et renvoie la distribution des temps.
4	`feat(frontend): dashboard complet`	Distribution des temps, mémoire, énergie, complexité théorique, recouvrement, diversité des genres.
5	`feat(frontend): historique des comparaisons de la session`	Revoir les comparaisons précédentes sans les relancer.

Terminé quand : *Tous les critères de comparaison du sujet sont visibles sur le dashboard.*

Vacances : tampon

19/12 – 03/01

Objectif : Pas de nouvelle fonctionnalité : rattraper le retard, corriger les bugs, relire le code des autres pour l'oral.

Phase 8 : Optimisation

04/01 – 12/01 · **v0.9.0**

Objectif : Rendre les recommandations plus rapides et plus sobres, et le prouver par des mesures avant / après.

#	Message de commit	Contenu
1	`perf(reco): top-n avec heapq au lieu d'un tri complet`	O(N log n) au lieu de O(N log N).
2	`perf(reco): vectorisation numpy de l'approche 1`	Genres en matrice binaire, Jaccard calculé par opérations matricielles.
3	`perf(reco): cache des recommandations`	functools.lru_cache sur (content_id, méthode, n).
4	`perf(frontend): images différées et anti-rebond de la recherche`	loading="lazy", debounce : moins de requêtes et moins de batterie côté client.
5	`test: tests automatisés des routes et des services`	pytest + client de test Flask (chapitre « tests automatisés » du cours).
6	`docs(optim): rapport de performances et calcul des coûts (rendu ~09/01)`	Tableau des mesures avant / après chaque optimisation.

Terminé quand : *Gains mesurés et documentés ; aucune régression (tests verts).*

Phase 9 : Version finale

10/01 – 19/01 · **v1.0.0**

Objectif : Livrer, documenter, présenter.

#	Message de commit	Contenu
1	`fix(frontend): corrections suite au test utilisateur`	Test par un camarade d'une autre équipe, retours consignés dans le rapport.
2	`docs: rapport final (rapport.md)`	Présentation, branding, public cible, stratégie de vente et commerciale, test utilisateur, licence.
3	`docs: vidéo de démonstration`	Lien dans le README ; commentaire audio recommandé.
4	`chore: version 1.0.0`	Tag v1.0.0 et push avant le 15/01.
5	`docs(optim): rapport d'optimisation global (rendu ~19/01)`	Complexité optimisée, points bloquants, différences entre les deux algorithmes.

Terminé quand : *Rendu final le 15/01, rapports le 19/01. Chaque membre sait expliquer toutes les parties à l'oral.*

5. Répartition proposée

Un domaine par personne pour avancer en parallèle (comme un blueprint par développeur dans le cours). Les noms sont à attribuer en équipe ; les rapports se rédigent à plusieurs.

Domaine	Responsabilités	Membre
Données & import	Scripts Jikan / TMDB, prepa_db, qualité des données (synopsis, genres).	à attribuer
Backend API & métriques	Blueprints, RecoLog, /api/metrics, /api/bench, mesures mémoire / énergie.	à attribuer
Approche 1	Filtrage par contenu, justification des résultats, tests, complexité.	à attribuer
Approche 2	Embeddings, similarité cosinus, repli TF-IDF, tests, complexité.	à attribuer
Front & branding	Pages, dashboard Chart.js, identité visuelle, test utilisateur.	à attribuer

- **Rapports d'optimisation** : responsables des approches 1 et 2 + backend métriques.
- **Branding, communication, rapport final** : responsable front + toute l'équipe pour la stratégie commerciale.
- **Vidéo et oral** : chacun présente sa partie et doit pouvoir répondre sur celles des autres.

6. Risques et parades

Risque	Parade
Limite de requêtes Jikan / indisponibilité	Import une seule fois, résultat en base ; pause entre requêtes ; dataset Kaggle en secours.
Clé TMDB exposée sur GitHub	Fichier .env ignoré par git, .env.example commité à la place.
sentence-transformers trop lourd (PyTorch)	Embeddings calculés une fois sur un seul poste et partagés en .npy ; repli TF-IDF.
Conflits git et retard d'intégration	Petites branches courtes, merge fréquent sur main, une revue par PR.
Un membre ne sait pas expliquer une partie à l'oral	Relecture croisée obligatoire des PR, démo interne avant chaque rendu.
Retard sur le 18/11	La v0.5.0 n'exige qu'une reco : l'approche 2 peut glisser en phase 5 sans risque.